GUÍA DEL ALUMNO

Quién guarda el dato. Quién lo puede cambiar.

La Tienda de cursos, partida en seis piezas: **props tipadas, emits, defineModel, slots, slots con alcance y composables.**

Manejo del estado local

Vue 3.5 · Composition API · TypeScript

Oscar Alvarado

Frontend Tech Lead

Universidad San Martín de Porres

Lima, Perú

ANTES DE EMPEZAR · QUÉ VAS A ENCONTRAR EN ESTA GUÍA

La misma Tienda, en seis piezas

Seguimos con **la Tienda de cursos** de la sesión 2. Funciona, pero es un solo `Tienda.vue` de 156 líneas que se encarga de todo. Hoy **no le agregamos funciones: la partimos en piezas** que se hablan entre sí y se pueden reutilizar. Al final se ve y funciona igual. Antes de cada ejemplo está la **teoría** de lo que se usa: qué es cada herramienta, para qué sirve y un diagrama de cómo funciona, el mismo de las slides. Después, los pasos del ejemplo en orden, diciendo **dónde** va cada cosa. El código completo de cada pieza está al final de su bloque.

1
Comunicación
ejemplos 1–2 · props · emits · defineModel

2
Slots
ejemplos 3–4 · slots · slots con alcance

3
Composables
ejemplos 5–6 · useCarrito · useId · useTemplateRef

Las seis piezas

- ✓ 1. TarjetaCurso · props tipadas y emits
- ✓ 2. ItemCarrito · defineModel
- ✓ 3. Panel · slots y slots con nombre
- ✓ 4. ListaEstados · slots con alcance
- ✓ 5. useCarrito · composables
- ✓ 6. CampoTexto · useId y useTemplateRef

A dónde vamos

```
src/
├── cursos.ts
├── App.vue
├── components/
│   ├── Tienda.vue          ← hoy: 156 líneas que hacen todo
│   ├── TarjetaCurso.vue    ej. 1 · props y emits
│   ├── ItemCarrito.vue     ej. 2 · defineModel
│   ├── Panel.vue           ej. 3 · slots
│   ├── ListaEstados.vue    ej. 4 · scoped slots
│   └── CampoTexto.vue      ej. 6 · useId
└── composables/
    ├── useLocalStorage.ts ej. 5
    └── useCarrito.ts       ej. 5
```

Una buena pieza se describe en una frase **sin usar "y"**: "muestra un curso". La Tienda de hoy necesita seis.

Lo que necesitas instalado

- **Node.js 22.18 o superior** (comprueba con `node -v`).
- **VS Code** con la extensión **Vue (Official)**: hoy miraremos mucho los subrayados rojos y los tipos al pasar el mouse. Y las **DevTools** del navegador (F12).

PUNTO DE PARTIDA · LA TIENDA DE LA SESIÓN 2 5 MIN

Sigue en tu proyecto, o empieza uno nuevo

terminal

```
# A · Tienes la Tienda de la sesión 2: sigue en ese proyecto
cd reactividad
npm run dev

# B · No la tienes: proyecto nuevo y pega los 4 archivos del chat
npm create vue@latest componentes -- --ts --bare
cd componentes
npm install
npm run dev
```

Camino A: si tu Tienda de la sesión 2 funciona, sigue en ese proyecto. **Camino B:** si no, crea uno nuevo y copia estos cuatro archivos. `cursos.ts` y `App.vue` son los mismos de la sesión 2.

src/cursos.ts

```ts
export interface Curso {
  id: number
  titulo: string
  precio: number
  categoria: 'frontend' | 'backend' | 'datos'
}

export const CURSOS: Curso[] = [
  { id: 1, titulo: 'Vue desde cero', precio: 120, categoria: 'frontend' },
  { id: 2, titulo: 'TypeScript práctico', precio: 90, categoria: 'frontend' },
  { id: 3, titulo: 'Node y Express', precio: 110, categoria: 'backend' },
  { id: 4, titulo: 'Bases de datos SQL', precio: 100, categoria: 'datos' },
  { id: 5, titulo: 'APIs con NestJS', precio: 150, categoria: 'backend' },
  { id: 6, titulo: 'Python para datos', precio: 130, categoria: 'datos' },
]
```

src/App.vue

```vue
<script setup lang="ts">
// La Tienda de cursos: la construimos paso a paso en clase
import Tienda from './components/Tienda.vue'
</script>

<template>
  <main>
    <Tienda />
  </main>
</template>

<style>
body {
  font-family: system-ui, sans-serif;
  line-height: 1.5;
  max-width: 720px;
  margin: 2rem auto;
  padding: 0 1rem;
}
input,
select,
textarea,
button {
  font: inherit;
  padding: 0.25rem 0.5rem;
}
section {
  margin-block: 1.5rem;
}
</style>
```

src/components/ItemCarrito.vue · punto de partida

```vue
<script setup lang="ts">
import { watch } from 'vue'

const {
  titulo,
  precio,
  cantidad = 1,
} = defineProps<{
  titulo: string
  precio: number
  cantidad?: number
}>()

watch(
  () => cantidad,
  (nuevo, anterior) => {
    console.log(`${titulo}: ${anterior} → ${nuevo}`)
  },
)
</script>

<template>
  <span>{{ titulo }} × {{ cantidad }} = S/ {{ (precio * cantidad).toFixed(2) }}</span>
</template>
```

src/components/Tienda.vue · punto de partida

```vue
<script setup lang="ts">
import {
  ref,
  reactive,
  toRefs,
  computed,
  watch,
  watchEffect,
  useTemplateRef,
  type Directive,
} from 'vue'
import { CURSOS, type Curso } from '@/cursos'
import ItemCarrito from './ItemCarrito.vue'

interface Linea {
  curso: Curso
  cantidad: number
}

const CLAVE = 'tienda-carrito'
const guardado = localStorage.getItem(CLAVE)
const carrito = ref<Linea[]>(guardado ? JSON.parse(guardado) : [])

function agregar(curso: Curso) {
  const linea = carrito.value.find((l) => l.curso.id === curso.id)
  if (linea) linea.cantidad++
  else carrito.value.push({ curso, cantidad: 1 })
}
const buscador = useTemplateRef('buscador')
```

```
function vaciar() {
  carrito.value = []
  buscador.value?.focus()
}

const cupon = ref('')
const mensaje = ref('')
const descuento = ref(0)

const cantidadTotal = computed(() => carrito.value.reduce((suma, l) => suma + l.cantidad, 0))
const subtotal = computed(() =>
  carrito.value.reduce((suma, l) => suma + l.curso.precio * l.cantidad, 0),
)
const igv = computed(() => subtotal.value * 0.18)
const total = computed(() => (subtotal.value + igv.value) * (1 - descuento.value))

watch(
  carrito,
  (nuevo) => {
    localStorage.setItem(CLAVE, JSON.stringify(nuevo))
  },
  { deep: true },
)

watch(cupon, (nuevo, anterior) => {
  console.log(`cupón: "${anterior}" → "${nuevo}"`)
  mensaje.value = 'Validando…'
  setTimeout(() => {
    const valido = nuevo === 'VUE2026'
    descuento.value = valido ? 0.1 : 0
    mensaje.value = valido ? '✅ 10% de descuento' : '❌ Cupón no válido'
  }, 800)
})

watchEffect(() => {
  document.title = `Tienda (${cantidadTotal.value})`
})

const cargando = ref(true)
const cursos = ref<Curso[]>([])

setTimeout(() => {
  cursos.value = CURSOS
  cargando.value = false
}, 1500)

const busqueda = ref('')
const orden = ref<'titulo' | 'precio'>('titulo')

const visibles = computed(() => {
  const texto = busqueda.value.toLowerCase()
  return cursos.value
    .filter((c) => c.titulo.toLowerCase().includes(texto))
    .sort((a, b) =>
      orden.value === 'precio' ? a.precio - b.precio : a.titulo.localeCompare(b.titulo),
    )
})

const abierto = ref<number | null>(null)

function alternar(id: number) {
  abierto.value = abierto.value === id ? null : id
}
```

```
const cliente = reactive({ nombre: '', email: '' })
const { nombre } = toRefs(cliente)

const vFocus: Directive<HTMLElement> = {
  mounted: (el) => el.focus(),
}
</script>

<template>
  <h1>Tienda de cursos</h1>
  <p>Pedido de {{ nombre }} · {{ cantidadTotal }} cursos</p>

  <!-- Alumno -->
  <section>
    <h2>Alumno</h2>
    <input v-focus v-model.trim="cliente.nombre" placeholder="Nombre" />
    <input v-model="cliente.email" placeholder="Email" />
  </section>

  <!-- Catálogo -->
  <section>
    <h2>Catálogo</h2>
    <input ref="buscador" v-model="busqueda" placeholder="Buscar curso…" />
    <select v-model="orden">
      <option value="titulo">Por título</option>
      <option value="precio">Por precio</option>
    </select>

    <p v-if="cargando">Cargando cursos…</p>
    <p v-else-if="visibles.length === 0">No hay cursos con "{{ busqueda }}".</p>
    <template v-else>
      <p>{{ visibles.length }} cursos</p>
      <ul>
        <li v-for="curso in visibles" :key="curso.id">
          <button @click="alternar(curso.id)">{{ curso.titulo }}</button>
          <button @click="agregar(curso)">Agregar</button>
          <p v-show="abierto === curso.id">{{ curso.categoria }} · S/ {{ curso.precio }}</p>
        </li>
      </ul>
    </template>
  </section>

  <!-- Carrito -->
  <section>
    <h2>Carrito <button @click="vaciar">Vaciar</button></h2>
    <ul>
      <li v-for="linea in carrito" :key="linea.curso.id">
        <input v-model.number="linea.cantidad" size="2" />
        <ItemCarrito
          :titulo="linea.curso.titulo"
          :precio="linea.curso.precio"
          :cantidad="linea.cantidad"
        />
      </li>
    </ul>
    <input v-model.lazy="cupon" placeholder="Cupón de descuento" />
    <p>{{ mensaje }}</p>
    <p>Subtotal: S/ {{ subtotal.toFixed(2) }}</p>
    <p>IGV (18%): S/ {{ igv.toFixed(2) }}</p>
    <p><strong>Total: S/ {{ total.toFixed(2) }}</strong></p>
  </section>
</template>
```

BLOQUE 1 · COMUNICACIÓN ENTRE COMPONENTES 23 MIN

Los datos bajan, los avisos suben

CANAL	DIRECCIÓN	PARA QUÉ
`defineProps`	padre → hijo	pasarle datos al hijo: la orden del pedido
`defineEmits`	hijo → padre	avisar que pasó algo: el timbre de la cocina
`defineModel`	los dos sentidos	compartir un dato con `v-model`

CONCEPTO props tipadas

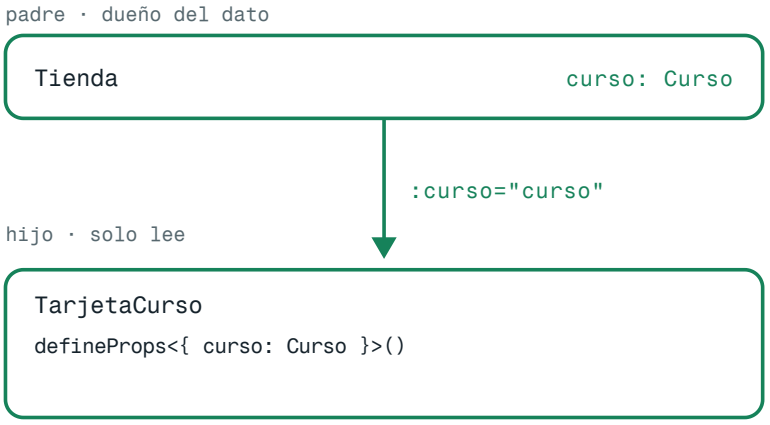
QUÉ ES

El **contrato** de un componente: qué datos necesita del padre para funcionar.

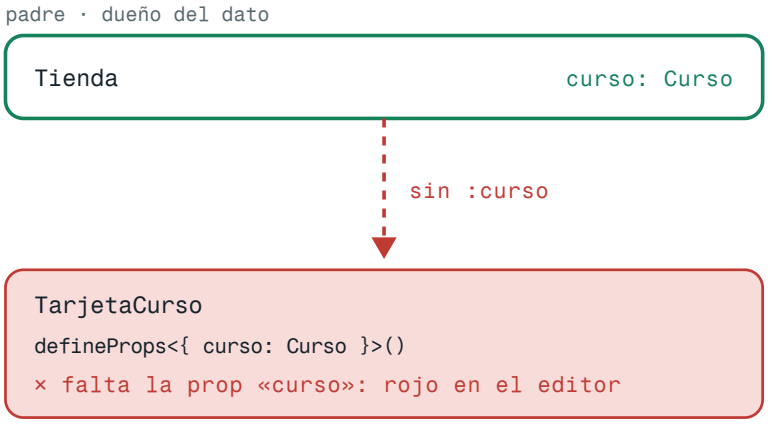
PARA QUÉ SE USA

Que un hijo reciba un dato (un curso, un precio) sin ser su dueño, y que el editor avise si falta o es de otro tipo.

Paso 1 de 4



Paso 2 de 4



Cómo funciona, paso a paso

1. La Tienda tiene el `curso` y se lo baja a `TarjetaCurso` con `:curso="curso"`. Los datos **bajan**.

2. El tipo es el contrato: si el padre olvida la prop, el editor la marca en rojo **antes** de ejecutar.
3. Las props son de **solo lectura**: el hijo no puede reemplazarlas. El dueño del dato es el padre.
4. Una prop opcional lleva `?` y su valor por defecto se escribe con `=`, como en la sesión 2.

REGLA

Un hijo **nunca cambia** lo que recibe. Si algo tiene que cambiar, avisa al dueño del dato: el siguiente tema.

CONCEPTO `defineEmits`

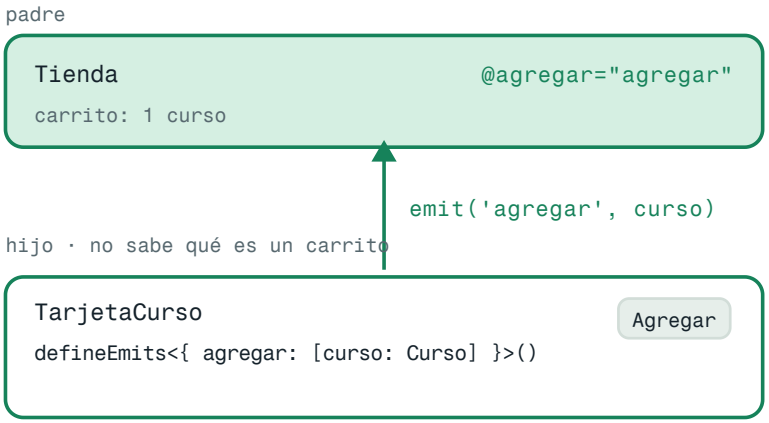
QUÉ ES

Los **avisos** que un hijo puede mandarle al padre, con sus datos.

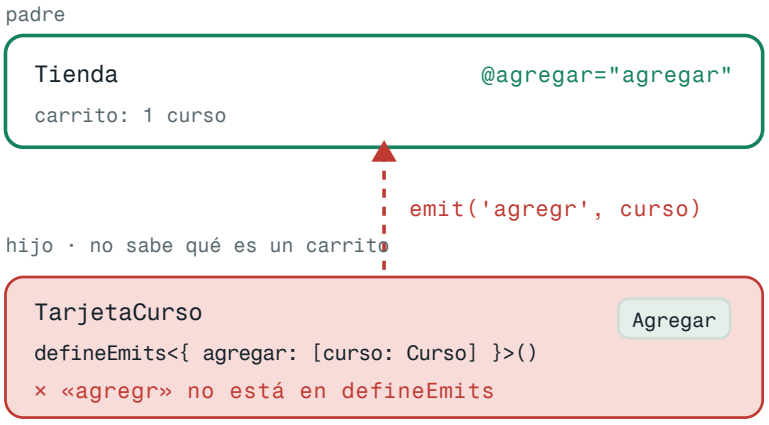
PARA QUÉ SE USA

Que el hijo diga «me agregaron» sin saber qué es un carrito: quien decide qué hacer es el padre.

Paso 3 de 4



Paso 4 de 4



Cómo funciona, paso a paso

1. `defineEmits` declara los avisos del hijo y los datos que llevan: `agregar: [curso: Curso]`.
2. El clic llama a `emit('agregar', curso)`: el aviso **sube** con el curso. Es el timbre de la cocina.

3. El padre escucha con `@agregar`, igual que un `@click`, y **decide** qué hacer: aquí, meterlo en el carrito.
4. Un nombre mal escrito, `emit('agregr')`, es un error de TypeScript, no un bug silencioso.

REGLA

Los datos **bajan** por props; los avisos **suben** por emits. Lo que solo le importa a un componente, como `abierto`, vive en él.

Ejemplo 1 de 6 · TarjetaCurso: props tipadas y emits

· 8 min

Cada curso del catálogo pasa a ser su propio componente: recibe el curso y avisa cuando lo agregan.

1. **Archivo nuevo** — `src/components/TarjetaCurso.vue` con `defineProps<{ curso: Curso }>()` que muestra el título y el detalle. **Tienda · Template** — el contenido de cada `<li>` del catálogo pasa a `<TarjetaCurso :curso="curso" />`. Borra `:curso`: el editor lo marca en rojo.
2. **TarjetaCurso** — `const abierto = ref(false)`; el título lo alterna y el detalle usa `v-show`. **Tienda · Script** — borra `abierto` y `alternar`: ahora se pueden abrir varios detalles a la vez.
3. **TarjetaCurso** — `defineEmits<{ agregar: [curso: Curso] }>()` y un botón que hace `emit('agregar', curso)`. **Tienda · Template** — `@agregar="agregar"`.
4. **TarjetaCurso** — una prop opcional `enCarrito?: number` con `= 0` por defecto. **Tienda** — una función `cantidadEn(id)` y `:en-carrito="cantidadEn(curso.id)"`.

CONCEPTO

defineModel

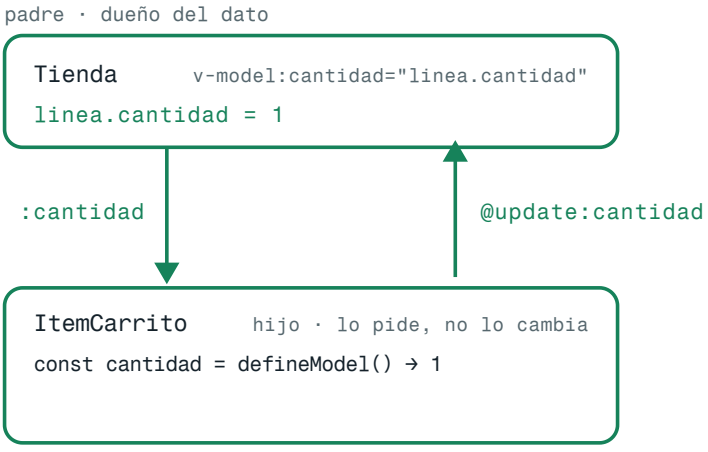
QUÉ ES

`v-model` en un componente: una **prop que baja y un evento que sube**, declarados en una sola línea.

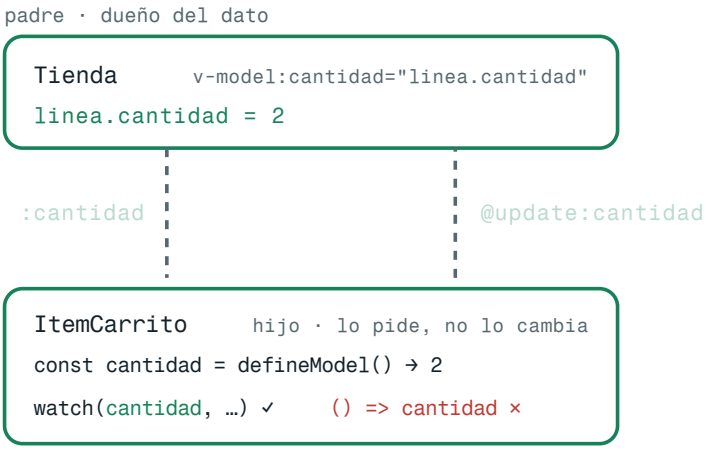
PARA QUÉ SE USA

Que el hijo **edite** un dato del padre, como la cantidad de una línea del carrito, sin ser su dueño.

Paso 1 de 5



Paso 5 de 5



Cómo funciona, paso a paso

1. `v-model:cantidad` es un atajo: pasa la prop `cantidad` y escucha `update:cantidad`.
2. El hijo escribe `cantidad++`. No cambia el dato: **pide** el cambio con el evento.
3. El padre, que es el dueño, actualiza `linea.cantidad`.
4. El valor nuevo **baja** por la prop: el hijo y el contador de la Tienda ven lo mismo.
5. `defineModel` devuelve un `ref`: `watch(cantidad, …)` funciona directo, sin el getter de la sesión 2.

REGLA

Si el hijo **edita** el dato, `defineModel`. Si solo lo **muestra**, una prop.

Ejemplo 2 de 6 · ItemCarrito con defineModel

· 6 min

La línea del carrito se encarga de su cantidad y avisa cuando la quieren quitar.

1. **ItemCarrito** — `defineEmits<{ quitar: [] }>()` y un botón "Quitar". **Tienda** — una función `quitar(id)` con `filter` y `@quitar="quitar(linea.curso.id)"`. `[]` significa que el aviso no lleva datos.
2. **ItemCarrito** — la prop `cantidad` pasa a `defineModel<number>('cantidad', { required: true })` y el input de cantidad se muda adentro. El `watch` ya no necesita getter: `watch(cantidad, …)`. **Tienda** — `v-model:cantidad="linea.cantidad"`.
3. **ItemCarrito** — botones `−` y `+` con `cantidad--` y `cantidad++`: la Tienda se entera sola.

El mismo atajo, escrito a mano y con `defineModel`:

```vue
<!-- Esto… -->
<ItemCarrito v-model:cantidad="linea.cantidad" />

<!-- …es un atajo de esto: una prop + un evento -->
<ItemCarrito
  :cantidad="linea.cantidad"
  @update:cantidad="(nueva) => (linea.cantidad = nueva)"
/>
```

Las dos piezas del bloque 1

src/components/TarjetaCurso.vue

```vue
<script setup lang="ts">
import { ref } from 'vue'
import type { Curso } from '@/cursos'

const { curso, enCarrito = 0 } = defineProps<{
  curso: Curso
  enCarrito?: number
}>()

const emit = defineEmits<{
  agregar: [curso: Curso]
}>()

const abierto = ref(false)
</script>

<template>
  <article>
    <button @click="abierto = !abierto">{{ curso.titulo }}</button>
    <button @click="emit('agregar', curso)">Agregar</button>
    <span v-if="enCarrito > 0"> · {{ enCarrito }} en el carrito</span>
    <p v-show="abierto">{{ curso.categoria }} · S/ {{ curso.precio }}</p>
  </article>
</template>
```

src/components/ItemCarrito.vue

```vue
<script setup lang="ts">
import { watch } from 'vue'

const { titulo, precio } = defineProps<{
  titulo: string
  precio: number
}>()

const cantidad = defineModel<number>('cantidad', { required: true })

const emit = defineEmits<{
  quitar: []
}>()

watch(cantidad, (nuevo, anterior) => {
  console.log(`${titulo}: ${anterior} → ${nuevo}`)
})
</script>

<template>
  <button :disabled="cantidad <= 1" @click="cantidad--">-</button>
  <input v-model.number="cantidad" size="2" />
  <button @click="cantidad++">+</button>
  <span>{{ titulo }} × {{ cantidad }} = S/ {{ (precio * cantidad).toFixed(2) }}</span>
  <button @click="emit('quitar')">Quitar</button>
</template>
```

BLOQUE 2 · SLOTS Y COMPONENTES REUTILIZABLES 21 MIN

Una prop pasa un dato. Un slot pasa un pedazo de template

SI PASAS…	USA	EN LA TIENDA
un texto o un dato	prop	`titulo="Carrito"`
HTML o componentes	slot	los inputs del Alumno
contenido para varias zonas	slots con nombre	`#acciones`, `#pie`
contenido que usa datos del hijo	slot con alcance	`#item="{ item }"`

DISEÑAR UN COMPONENTE REUTILIZABLE

¿Qué es siempre igual? La estructura: va en el componente. **¿Qué cambia cada vez?** Los datos van por props; el contenido, por slots. **¿Qué tiene que avisar?** Lo que pasa adentro sube por emits.

CONCEPTO slots

QUÉ ES

Un **hueco** en el template de un componente que el padre rellena con lo que quiera.

PARA QUÉ SE USA

Reutilizar la **estructura** (un marco, una tarjeta, un modal) y cambiar el contenido cada vez.

Paso 2 de 5

Panel.vue · el marco



Paso 4 de 5

Panel.vue · el marco



Cómo funciona, paso a paso

1. `Panel` es un marco: el título es un texto (una **prop**) y el contenido, un hueco: `<slot />`.
2. Todo lo que escribes entre `<Panel>` y `</Panel>` cae en ese hueco. Mismo marco, otra foto.
3. Un componente puede tener varios huecos, cada uno con su nombre: `<slot name="acciones" />` se rellena con `#acciones`.
4. `$slots.pie` dice si el padre rellenó un hueco: el Alumno no tiene pie, así que no se dibuja un `<footer>` vacío.
5. Lo que pones dentro del `<slot>` es el contenido por defecto: se ve solo si el padre no manda nada.

REGLA

Una prop pasa un **dato**; un slot pasa un **pedazo de template**. El componente pone la estructura; quien lo usa, el contenido.

Ejemplo 3 de 6 · Panel: slots y slots con nombre

· 6 min

Las tres secciones de la Tienda comparten un mismo marco: título, acciones, contenido y pie.

1. **Archivo nuevo** — `src/components/Panel.vue` con la prop `titulo` y un `<slot />`. **Tienda** — la sección Alumno pasa a `<Panel titulo="Alumno">`.
2. **Panel** — `<slot name="acciones" />` en el encabezado. **Tienda** — Catálogo y Carrito también pasan a `<Panel>`; el botón "Vaciar" va en `<template #acciones>`.
3. **Panel** — `<footer v-if="$slots.pie">` con `<slot name="pie" />`. **Tienda** — los totales van en `<template #pie>`: solo el Carrito tiene pie.
4. **Panel** — contenido por defecto dentro de `<slot>`: se ve si el padre no manda nada.

CONCEPTO

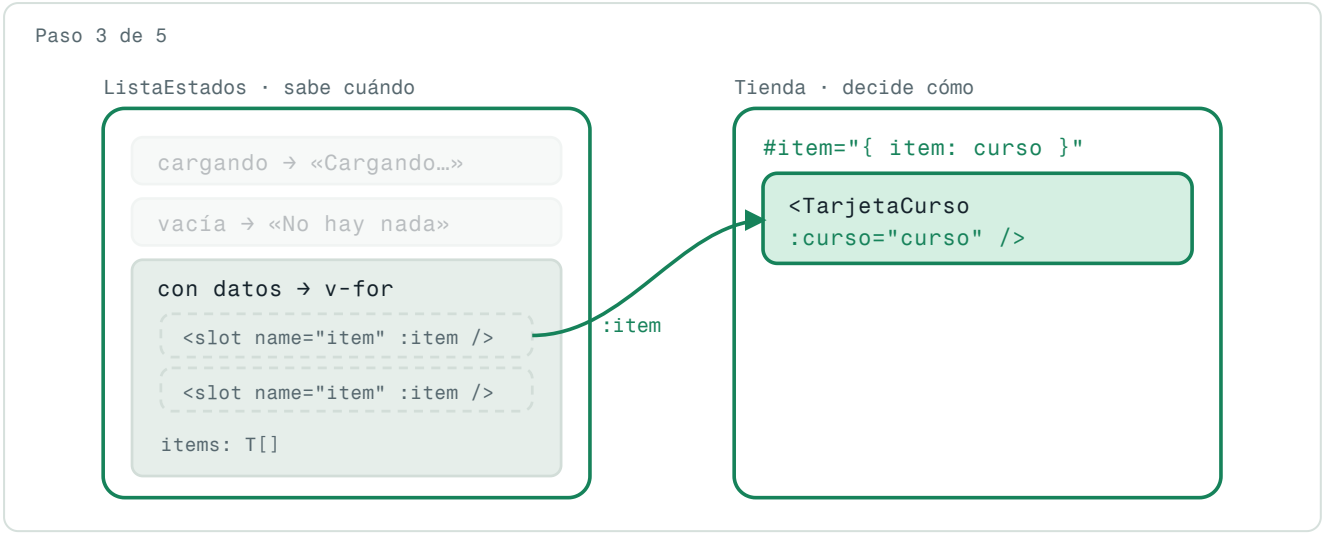
slots con alcance

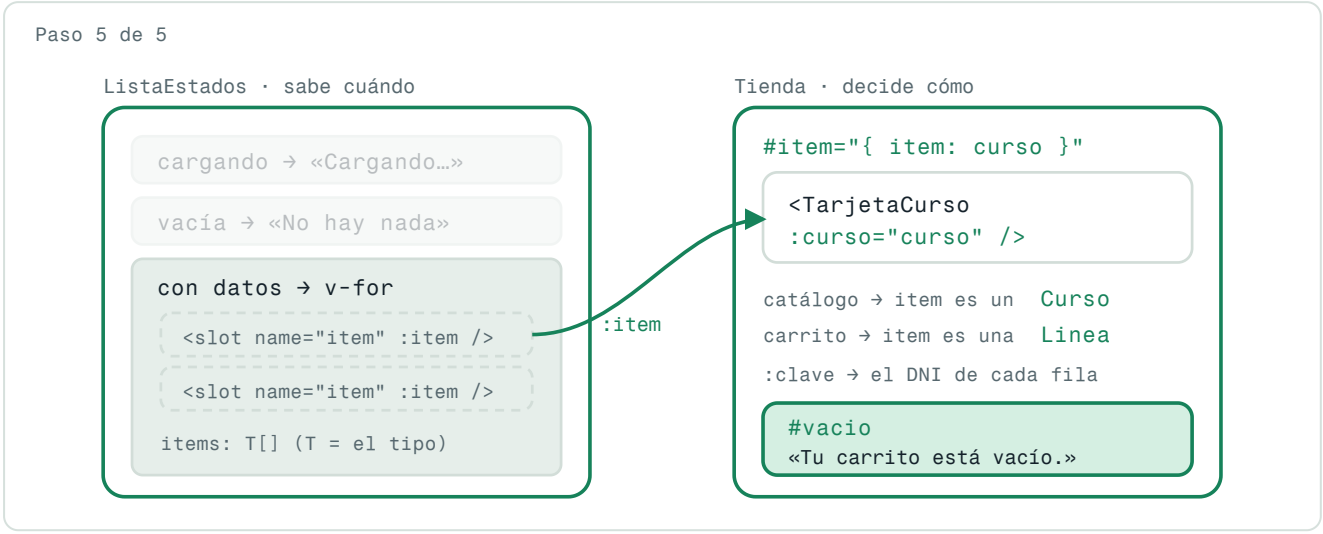
QUÉ ES

Un slot que además **le presta datos** al padre para que dibuje cada elemento.

PARA QUÉ SE USA

Listas, tablas y menús donde el componente sabe **cuándo** dibujar y quien lo usa decide **cómo** se ve.





Cómo funciona, paso a paso

1. Toda lista tiene tres estados: cargando, vacía y con datos. `ListaEstados` los escribe una sola vez.
2. En «con datos», un `v-for` deja un hueco por fila: `<slot name="item" :item="item" />`.
3. La lista **presta** cada `item` y el padre lo recibe con `#item="{ item: curso }"` para dibujarlo. Es un molde.
4. Con `generic="T"` la lista sirve para cualquier tipo sin perderlo: con el catálogo, `item` es un `Curso`; con el carrito, una `Linea`.
5. El estado vacío también lo decide el padre con un slot `#vacio`: cada lista pone su mensaje.

REGLA

Toda lista tiene tres estados. Escríbelos **una vez**, en un componente genérico, y reúsalo con cualquier tipo de elemento.

Ejemplo 4 de 6 · ListaEstados: slots con alcance

· 9 min

Toda lista tiene tres estados: cargando, vacía y con datos. Los escribimos **una vez**, en un componente, y lo usamos para el catálogo y para el carrito.

1. **Archivo nuevo** — `src/components/ListaEstados.vue` con `generic="T"`, las props `items: T[]`, `clave: (item: T) => number` y `cargando?`, y `<slot name="item" :item="item" />` dentro del `v-for`.
2. **Tienda · Catálogo** — el `v-if` / `v-else-if` / `v-else` se reemplaza por `<ListaEstados>` con `<template #item="{ item: curso }">`. Pasa el mouse sobre `curso`: es un `Curso`.
3. **ListaEstados** — el mensaje de vacío pasa a `<slot name="vacio">`. **Tienda** — cada lista pone el suyo, y el Carrito también usa `<ListaEstados>`: ahí `item` es una `Linea`.
4. **Tienda · Catálogo** — el contador de cursos va en `#acciones` del panel.

Las dos piezas del bloque 2

src/components/Panel.vue

```vue
<script setup lang="ts">
defineProps<{
  titulo: string
}>()
</script>

<template>
  <section class="panel">
    <header>
      <h2>{{ titulo }}</h2>
      <slot name="acciones" />
    </header>

    <slot>
      <p>Nada por aquí todavía.</p>
    </slot>

    <footer v-if="$slots.pie">
      <slot name="pie" />
    </footer>
  </section>
</template>

<style scoped>
.panel {
  border: 1px solid #d0d7de;
  border-radius: 8px;
  padding: 0 1rem 1rem;
}
header {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
}
footer {
  border-top: 1px solid #d0d7de;
  margin-top: 1rem;
}
</style>
```

src/components/ListaEstados.vue

```vue
<script setup lang="ts" generic="T">
const {
  items,
  clave,
  cargando = false,
} = defineProps<{
  items: T[]
  clave: (item: T) => number
  cargando?: boolean
}>()
</script>

<template>
  <p v-if="cargando">Cargando…</p>
  <slot v-else-if="items.length === 0" name="vacio">
    <p>No hay nada que mostrar.</p>
  </slot>
  <ul v-else>
    <li v-for="item in items" :key="clave(item)">
      <slot name="item" :item="item" />
    </li>
  </ul>
</template>
```

BLOQUE 3 · COMPOSABLES 22 MIN

Un componente reutiliza lo que se ve. Un composable, lo que se hace

QUIERO…	USO	EN LA TIENDA
reutilizar algo que se ve	un componente	`Panel`, `ListaEstados`
reutilizar lógica con estado	un composable	`useCarrito`
un elemento o componente del template	`useTemplateRef`	el buscador, el campo del nombre
un id único para un label	`useId`	`CampoTexto`

CONCEPTO composables

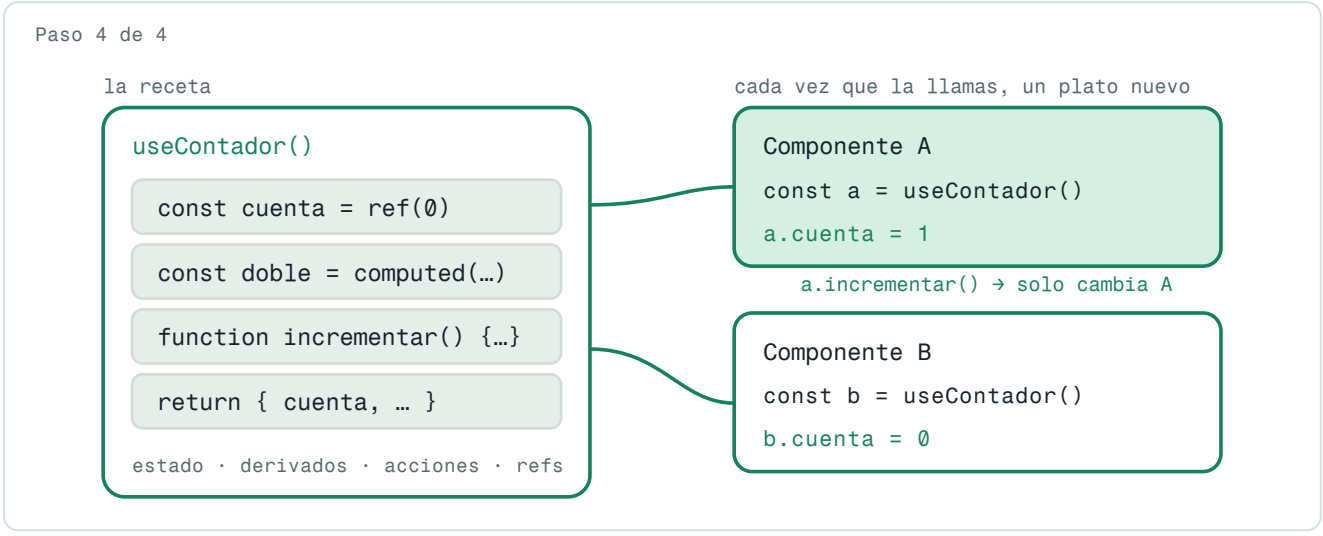
QUÉ ES

Una función `useAlgo()` que **agrupa estado y lógica** para reutilizarla.

PARA QUÉ SE USA

Compartir comportamiento con estado entre componentes: un contador, un carrito, guardar en el navegador.





Cómo funciona, paso a paso

1. Un composable es una receta con cuatro partes: **estado**, **derivados**, **acciones** y un `return` con `ref`s.
2. Un componente la usa: `const a = useContador()` y obtiene sus propios ingredientes.
3. Otro componente llama a la misma receta y obtiene **otro** plato, con su propio estado.
4. Incrementar en uno no toca al otro: dos llamadas, dos estados independientes. `useTemplateRef` y `useId` también son composables: los que trae Vue.

REGLA

Un componente reutiliza lo que **se ve**; un composable, lo que **se hace**. Cada llamada crea su **propio** estado.

CONCEPTO useCarrito · useLocalStorage

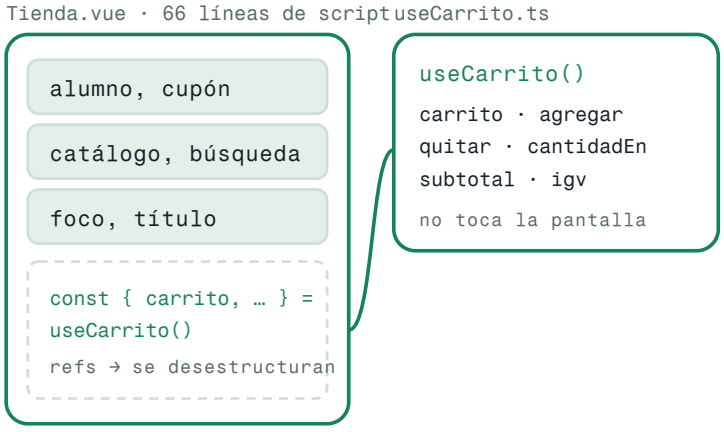
QUÉ ES

Sacar de un componente lo que **no toca la pantalla** y llevarlo a un composable.

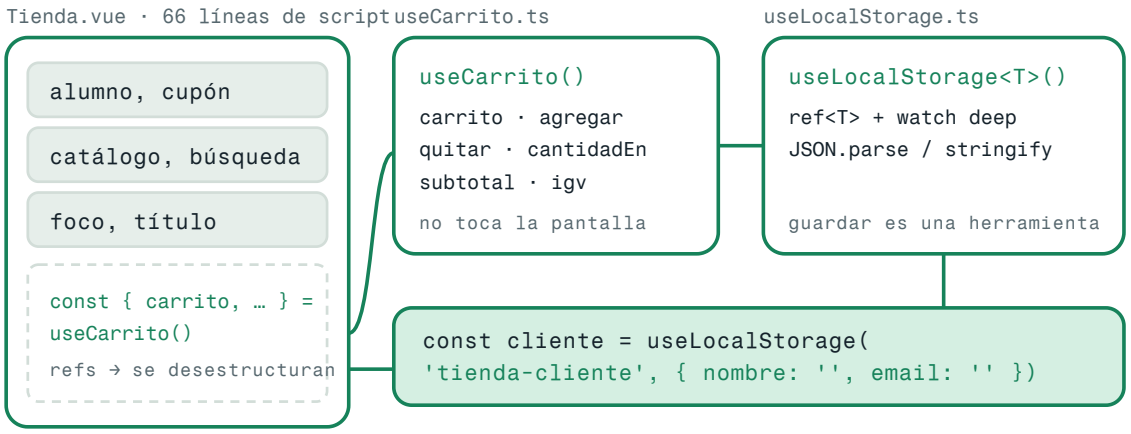
PARA QUÉ SE USA

Que el componente solo dibuje, y que la lógica se pueda leer y reusar por separado.

Paso 3 de 5



Paso 5 de 5



Cómo funciona, paso a paso

1. Hoy `Tienda.vue` mezcla pantalla (alumno, catálogo, foco) con la lógica del carrito: agregar, quitar, totales y guardar.
2. Se **corta y pega**: la lógica del carrito se muda a `useCarrito`. No se escribe nada nuevo.
3. La Tienda lo usa y su script baja de 107 a 66 líneas. Se desestructura sin miedo: lo que viaja son `ref`s y copias la caja.
4. `useCarrito` usa a su vez `useLocalStorage<T>`: guardar es una herramienta genérica.
5. El mismo composable guarda también al alumno: otro dato, mismo archivo.

> REGLA
>
> El componente **dibuja**. Lo que no depende de cómo se ve (agregar, quitar, guardar) va a un composable; el foco, que sí toca la pantalla, se queda.

Ejemplo 5 de 6 · useCarrito y useLocalStorage

· 8 min

La lógica del carrito sale de la Tienda a su propio archivo, y guardar en el navegador se vuelve reutilizable.

1. **Archivo nuevo** — `src/composables/useCarrito.ts`: **corta** de la Tienda `Linea`, el carrito y su `watch`, `agregar`, `quitar`, `cantidadEn` y los `computed`, y pégalos dentro de la función. `vaciar` solo vacía.
2. **Tienda · Script** — `const { carrito, agregar, … } = useCarrito()`. El botón "Vaciar" llama a `vaciarYBuscar()`, que vacía y enfoca el buscador.
3. **Archivo nuevo** — `src/composables/useLocalStorage.ts`, genérico. **useCarrito** — el carrito pasa a `useLocalStorage<Linea[]>('tienda-carrito', [])`.
4. **Tienda** — el alumno también se guarda: `useLocalStorage('tienda-cliente', { nombre: '', email: '' })`.

CONCEPTO

useId

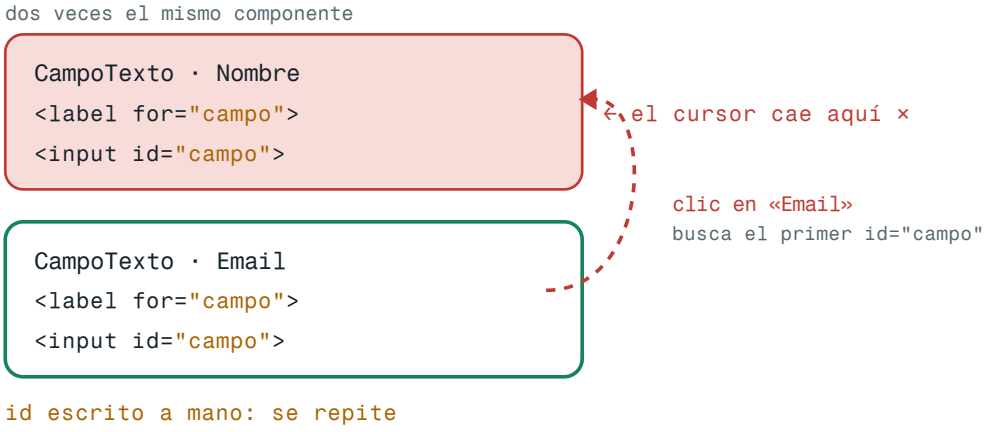
QUÉ ES

Un composable de Vue que genera un id **único y estable** por cada vez que se usa un componente.

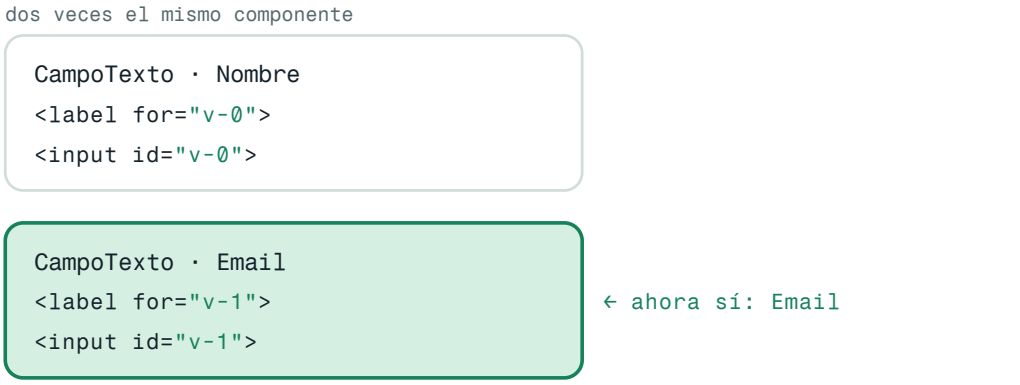
PARA QUÉ SE USA

Unir un `<label for>` con su input sin que dos usos del mismo componente se pisen.

Paso 2 de 4



Paso 4 de 4



Cómo funciona, paso a paso

1. Un `CampoTexto` con `id="campo"` escrito a mano, usado dos veces: **dos elementos con el mismo id**.
2. Clic en la etiqueta «Email»: el navegador busca el primer `id="campo"` y el cursor cae en **Nombre**.
3. `useId()` entrega un id distinto a cada uso: `v-0`, `v-1`.
4. Ahora cada etiqueta apunta a su propio input, y el id es el mismo en el servidor y en el navegador.

REGLA

Nunca escribas un `id` fijo dentro de un componente reutilizable. Tampoco `Math.random()`: cambia en cada carga y rompe el SSR.

CONCEPTO

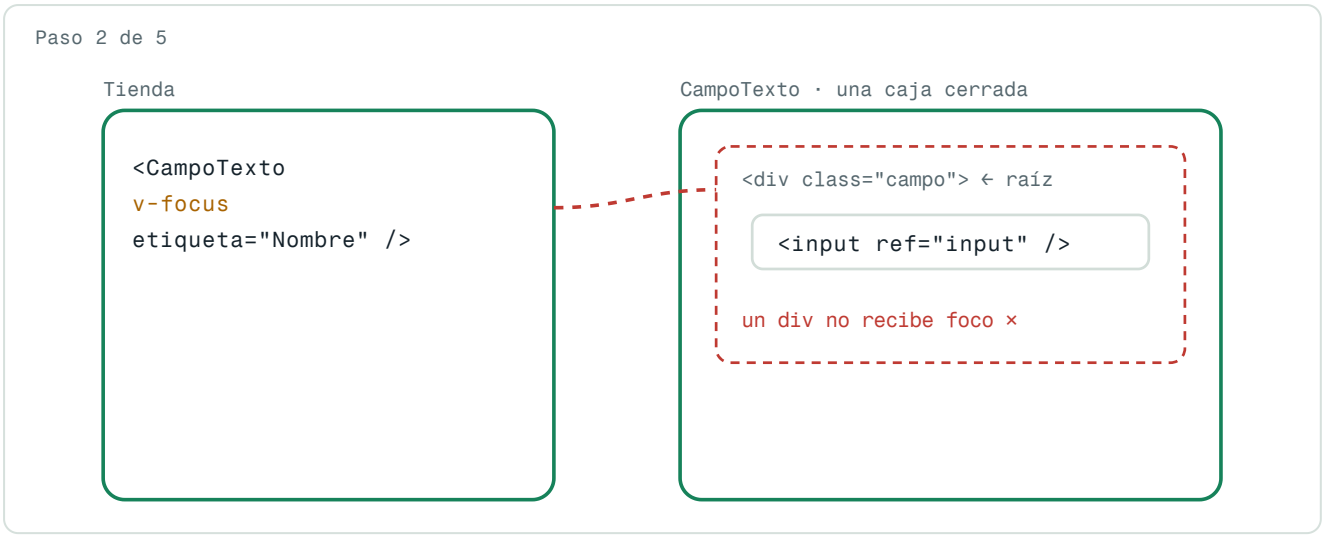
useTemplateRef · defineExpose

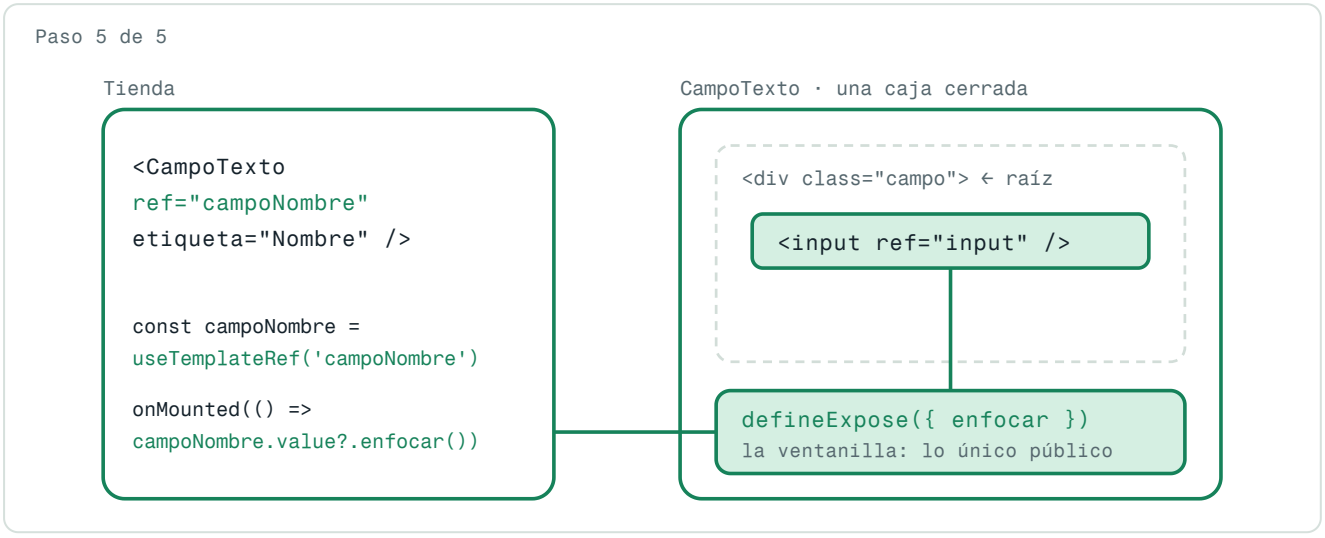
QUÉ ES

`useTemplateRef` agarra un elemento **o un componente** del template; `defineExpose` decide qué deja usar un componente desde afuera.

PARA QUÉ SE USA

Llamar a un método del hijo, como `enfocar()`, sin romper su encapsulamiento.





Cómo funciona, paso a paso

1. `v-focus` funcionaba sobre un `<input>`. Ahora lo ponemos sobre `CampoTexto`, un componente.
2. Una directiva en un componente se aplica a su **elemento raíz**: un `<div>`, que no recibe foco. No pasa nada.
3. Por defecto un componente es una caja cerrada: desde afuera no se ve el `<input>` de adentro.
4. `defineExpose({ enfocar })` abre una **ventanilla**: solo lo que pones ahí se puede usar desde afuera.
5. La Tienda agarra el componente con `useTemplateRef` y llama a `enfocar()` en `onMounted`: antes de montarse no existe.

REGLA

Una directiva sobre un componente cae en su **elemento raíz**. Si necesitas algo de adentro, que el componente lo **exponga**.

Ejemplo 6 de 6 · CampoTexto: useId y useTemplateRef

· 5 min

Los campos del alumno pasan a ser un componente con su etiqueta, y el nombre recibe el foco al cargar.

1. **Archivo nuevo** — `src/components/CampoTexto.vue` con la prop `etiqueta`, `defineModel<string>()` y un `<label for="campo">`. **Tienda · Alumno** — dos `<CampoTexto>`. Dos bugs: clic en la etiqueta "Email" enfoca Nombre, y `v-focus` dejó de funcionar.
2. **CampoTexto** — `const id = useId()` y `:for="id"` / `:id="id"`: cada campo tiene su propio id.
3. **CampoTexto** — `useTemplateRef('input')` y `defineExpose({ enfocar })`. **Tienda** — borra `vFocus`; `useTemplateRef('campoNombre')` y `onMounted(() => campoNombre.value?.enfocar())`.

Las piezas del bloque 3

src/composables/useLocalStorage.ts

```ts
import { ref, watch } from 'vue'

export function useLocalStorage<T>(clave: string, inicial: T) {
  const guardado = localStorage.getItem(clave)
  const valor = ref<T>(guardado ? JSON.parse(guardado) : inicial)

  watch(
    valor,
    (nuevo) => {
      localStorage.setItem(clave, JSON.stringify(nuevo))
    },
    { deep: true },
  )

  return valor
}
```

src/composables/useCarrito.ts

```ts
import { computed } from 'vue'
import type { Curso } from '@/cursos'
import { useLocalStorage } from './useLocalStorage'

export interface Linea {
  curso: Curso
  cantidad: number
}

export function useCarrito() {
  const carrito = useLocalStorage<Linea[]>('tienda-carrito', [])

  function agregar(curso: Curso) {
    const linea = carrito.value.find((l) => l.curso.id === curso.id)
    if (linea) linea.cantidad++
    else carrito.value.push({ curso, cantidad: 1 })
  }

  function quitar(id: number) {
    carrito.value = carrito.value.filter((l) => l.curso.id !== id)
  }

  function vaciar() {
    carrito.value = []
  }

  function cantidadEn(id: number) {
    return carrito.value.find((l) => l.curso.id === id)?.cantidad ?? 0
  }

  const cantidadTotal = computed(() => carrito.value.reduce((suma, l) => suma + l.cantidad,
   0))
  const subtotal = computed(() =>
    carrito.value.reduce((suma, l) => suma + l.curso.precio * l.cantidad, 0),
  )
  const igv = computed(() => subtotal.value * 0.18)

  return { carrito, agregar, quitar, vaciar, cantidadEn, cantidadTotal, subtotal, igv }
}
```

src/components/CampoTexto.vue

```vue
<script setup lang="ts">
import { useId, useTemplateRef } from 'vue'

defineProps<{
  etiqueta: string
}>()

const valor = defineModel<string>({ required: true })

const id = useId()
const input = useTemplateRef('input')

defineExpose({
  enfocar: () => input.value?.focus(),
})
</script>

<template>
  <div class="campo">
    <label :for="id">{{ etiqueta }}</label>
    <input :id="id" ref="input" v-model.trim="valor" />
  </div>
</template>

<style scoped>
.campo {
  display: flex;
  flex-direction: column;
  gap: 0.25rem;
  margin-bottom: 0.75rem;
}
</style>
```

La Tienda terminada

Del `Tienda.vue` que hacía todo queda el que **orquesta**: junta las piezas, guarda el cupón y el catálogo, y se encarga del foco.

src/components/Tienda.vue

```
<script setup lang="ts">
import { ref, computed, watch, watchEffect, useTemplateRef, onMounted } from 'vue'
import { CURSOS, type Curso } from '@/cursos'
import { useCarrito } from '@/composables/useCarrito'
import { useLocalStorage } from '@/composables/useLocalStorage'
import ItemCarrito from './ItemCarrito.vue'
import TarjetaCurso from './TarjetaCurso.vue'
import Panel from './Panel.vue'
import ListaEstados from './ListaEstados.vue'
import CampoTexto from './CampoTexto.vue'

const { carrito, agregar, quitar, vaciar, cantidadEn, cantidadTotal, subtotal, igv } =
  useCarrito()

const buscador = useTemplateRef('buscador')

function vaciarYBuscar() {
  vaciar()
  buscador.value?.focus()
}

const cupon = ref('')
const mensaje = ref('')
const descuento = ref(0)

const total = computed(() => (subtotal.value + igv.value) * (1 - descuento.value))

watch(cupon, (nuevo, anterior) => {
  console.log(`cupón: "${anterior}" → "${nuevo}"`)
  mensaje.value = 'Validando…'
  setTimeout(() => {
    const valido = nuevo === 'VUE2026'
    descuento.value = valido ? 0.1 : 0
    mensaje.value = valido ? '✅ 10% de descuento' : '❌ Cupón no válido'
  }, 800)
})

watchEffect(() => {
  document.title = `Tienda (${cantidadTotal.value})`
})

const cargando = ref(true)
const cursos = ref<Curso[]>([])

setTimeout(() => {
  cursos.value = CURSOS
  cargando.value = false
}, 1500)

const busqueda = ref('')
const orden = ref<'titulo' | 'precio'>('titulo')

const visibles = computed(() => {
```

```
  const texto = busqueda.value.toLowerCase()
  return cursos.value
    .filter((c) => c.titulo.toLowerCase().includes(texto))
    .sort((a, b) =>
      orden.value === 'precio' ? a.precio - b.precio : a.titulo.localeCompare(b.titulo),
    )
})

const cliente = useLocalStorage('tienda-cliente', { nombre: '', email: '' })

const campoNombre = useTemplateRef('campoNombre')

onMounted(() => campoNombre.value?.enfocar())
</script>

<template>
  <h1>Tienda de cursos</h1>
  <p>Pedido de {{ cliente.nombre }} · {{ cantidadTotal }} cursos</p>

  <Panel titulo="Alumno">
    <CampoTexto ref="campoNombre" v-model="cliente.nombre" etiqueta="Nombre" />
    <CampoTexto v-model="cliente.email" etiqueta="Email" />
  </Panel>

  <Panel titulo="Catálogo">
    <template #acciones>
      <span v-if="!cargando">{{ visibles.length }} cursos</span>
    </template>

    <input ref="buscador" v-model="busqueda" placeholder="Buscar curso…" />
    <select v-model="orden">
      <option value="titulo">Por título</option>
      <option value="precio">Por precio</option>
    </select>

    <ListaEstados :items="visibles" :clave="(curso) => curso.id" :cargando="cargando">
      <template #item="{ item: curso }">
        <TarjetaCurso :curso="curso" :en-carrito="cantidadEn(curso.id)" @agregar="agregar" />
      </template>
      <template #vacio>
        <p>No hay cursos con "{{ busqueda }}".</p>
      </template>
    </ListaEstados>
  </Panel>

  <Panel titulo="Carrito">
    <template #acciones>
      <button @click="vaciarYBuscar">Vaciar</button>
    </template>

    <ListaEstados :items="carrito" :clave="(linea) => linea.curso.id">
      <template #item="{ item: linea }">
        <ItemCarrito
          v-model:cantidad="linea.cantidad"
          :titulo="linea.curso.titulo"
          :precio="linea.curso.precio"
          @quitar="quitar(linea.curso.id)"
        />
      </template>
      <template #vacio>
        <p>Tu carrito está vacío.</p>
      </template>
    </ListaEstados>
```

```
    <input v-model.lazy="cupon" placeholder="Cupón de descuento" />
    <p>{{ mensaje }}</p>

    <template #pie>
      <p>Subtotal: S/ {{ subtotal.toFixed(2) }}</p>
      <p>IGV (18%): S/ {{ igv.toFixed(2) }}</p>
      <p><strong>Total: S/ {{ total.toFixed(2) }}</strong></p>
    </template>
  </Panel>
</template>
```

CIERRE · PARA SEGUIR

Ocho decisiones de diseño

NECESITO…	USO	EN LA TIENDA
pasarle datos a un hijo	`defineProps`	`curso` en TarjetaCurso
que el hijo avise algo al padre	`defineEmits`	`agregar`, `quitar`
que el hijo edite un dato del padre	`defineModel` + `v-model`	cantidad, campos del alumno
que el padre decida el contenido	slots	Panel
que el padre decida cómo se ve cada elemento	slot con alcance	ListaEstados
reutilizar lógica con estado	un composable	`useCarrito`, `useLocalStorage`
un elemento o componente del template	`useTemplateRef`	buscador, campo del nombre
un id único para un label	`useId`	CampoTexto

Para practicar

Escribe un composable `useCatalogo()` que se lleve de la Tienda `cargando`, `busqueda`, `orden` y `visibles`. Bonus: usa `CampoTexto` también para el buscador (pista: el buscador necesita su propio `enfocar`).

Glosario

TÉRMINO	EN UNA FRASE
Props tipadas	Las props escritas como una interfaz: el contrato del componente. Son de solo lectura.
Emit	Un aviso del hijo al padre, con o sin datos.
Dueño del dato	El componente que declaró el `ref`; solo él lo cambia.
`defineModel`	Una prop + el evento `update:`, en un `ref` que avisa al padre al escribirlo.
Slot	Un hueco en el template del hijo que rellena el padre.
Slot con alcance	Un slot que además presta datos al padre: un molde.
Componente genérico	Funciona con cualquier tipo sin perderlo: `generic="T"`.
Composable	Una función `useAlgo()` que agrupa estado y lógica para reutilizarla.
`defineExpose`	Lo que un componente deja usar desde afuera.
`useId`	Un id único y estable por cada uso del componente.

Para repasar

Props	vuejs.org
Eventos (emits)	vuejs.org
v-model en componentes	vuejs.org
Slots	vuejs.org
Composables	vuejs.org
useTemplateRef y useId	vuejs.org